

INGENIERÍA DE SISTEMAS

Tecnología Web II

ACTIVIDAD ACADÉMICA N°1

VISUALSYNC - SISTEMA WEB DE GESTIÓN DE PORTAFOLIOS, RESERVAS Y ENTREGA FOTOGRÁFICA

AUTORES

David Miguel Flores Medrano

Docente: Vladimir Wilmar Rojas Condori

Cochabamba-Bolivia

11-10-2026

Índice de contenido

1. Descripción del problema 1

2. Justificación de la solución 1

3. Objetivo general y objetivos específicos 2

3.1 Objetivo General 2

3.1.2 Objetivos Específicos 2

4. Usuarios y actores 2

5. requerimientos funcionales y no funcionales 3

5.1 requerimientos Funcionales (RF): 3

5.1.2 Requerimientos No Funcionales (RNF): 3

6. Modelado C4 4

Nivel 1: Diagrama de Contexto 4

Nivel 2: Diagrama de Contenedores 4

Nivel 3: Diagrama de Componentes 5

Nivel 4: Diagrama de Código (Clases) 7

7.Historias de Usuario 7

7.1 Estados del sistema 8

7.1.2 Módulo 0: Acceso 8

7.1.3 Módulo 1: Catálogo y Reservas 9

7.1.4 Módulo 2: Pagos y Validación QR 10

7.1.5 Módulo 3: Entrega de Material (Galerías) 11

7.1.6 Trabajo futuro (fuera del alcance actual) 12

8. Arquitectura propuesta 13

8.1 Capa de presentación (Cliente / Vistas) 13

8.1.2 Capa de lógica de aplicación (Servidor / Backend) 13

8.1.3 Capa de datos 14

9. Modelo de datos inicial 14

9.1 Tabla: Usuarios 14

9.1.2 Tabla: PaquetesServicio 15

9.1.3 Tabla: Reservas 16

9.1.4 Tabla: ArchivosGaleria 17

9.1.5 Relaciones (multiplicidad) 18

10. Prototipo de interfaces 18

10.1. Vista pública: Catálogo de servicios (HU-01) 18

10.1.2 Vista pública: Registro e inicio de sesión (HU-07) 19

10.1.3 Vista de cliente: Solicitud de reserva y pago (HU-02, HU-03) 21

10.1.4 Vista de cliente: Mis reservas (HU-03) 21

10.1.5 Vista de cliente: Galería privada y descargas (HU-06) 21

10.1.6 Vista de administrador: Panel de gestión (HU-04, HU-05) 22

11. Tecnologías seleccionadas y justificación 22

11.1.1 Frontend (capa de presentación) 22

11.1.2 Backend (lógica de aplicación) 23

11.1.3 Base de datos y almacenamiento 23

11.1.4 Control de versiones y herramientas..... 23

12. Estructura inicial de la API 24

13.1 Repositorio GitHub 31

13.1.1 Alojamiento del proyecto 31

13.1.2 Estrategia de ramas 31

13.1.3 Convención de commits 32

13.1.4 Archivos ignorados (. gitignore)..... 32

14. Conclusiones 32

INDICE DE ILUSTRACIONES

Ilustración 1: Nivel 1: Diagrama de Contexto 4

Ilustración 2: Nivel 2: Diagrama de Contenedores 5

Ilustración 3: Nivel 3: Diagrama de Componentes 6

Ilustración 4: Nivel 4: Diagrama de Código (Clases) 7

Ilustración 5: Vista pública: Catálogo de servicios..... 19

Ilustración 6: Vista pública: Inicio de Sesion..... 20

Ilustración 7: Vista pública: Registrarse..... 20

INDICE DE TABLAS

Tabla 1: Usuarios y Actores..... 2

Tabla 2: Estados del Sistema..... 8

Tabla 3: Registro e Inicio de Sesion 8

Tabla 4: Catálogo y Reservas 9

Tabla 5: Pagos y Validacion QR 10

Tabla 6: Entrega del Material 11

Tabla 7: trabajo para el futuro 12

Tabla 8: Usuarios 14

Tabla 9: Paquetes Servicios 15

Tabla 10: Revervas 16

Tabla 11: Archivos Galeria 17

Tabla 12: Estructura de carpetas 24

Tabla 13:de rutas iniciales 25

Tabla 14: Detalle de peticiones y respuestas 28

1. Descripción del problema

En el rubro de la fotografía profesional y cobertura de eventos sociales (bodas, quinceañeros, sesiones corporativas), la gestión de clientes, el agendamiento de fechas y la entrega del material final suelen manejarse de forma fragmentada e informal a través de aplicaciones de mensajería instantánea y enlaces temporales en la nube.

Esto provoca tres problemas críticos:

- Desorganización en reservas y pagos: Pérdida de seguimiento sobre la disponibilidad de fechas, paquetes contratados y comprobantes de anticipos enviados por chat.
- Vulnerabilidad en la entrega de material: Dificultad para controlar que el cliente solo descargue las fotografías en alta resolución una vez que haya liquidado el saldo total del servicio.
- Falta de presencia centralizada: Ausencia de un catálogo estructurado donde los clientes potenciales puedan revisar el portafolio y los paquetes de precios sin depender de la respuesta manual inmediata del fotógrafo.

2. Justificación de la solución

El desarrollo de VisualSync centraliza todo el flujo de negocio de un estudio fotográfico en una única plataforma web moderna.

- Operativa: Automatiza la exhibición de paquetes y la recepción de solicitudes de reserva, reduciendo el tiempo administrativo del fotógrafo.
- Financiera y de Seguridad: Implementa una regla de negocio estricta donde las galerías privadas muestran únicamente previsualizaciones protegidas (con marca de agua o baja resolución) hasta que el administrador valida manualmente el comprobante de pago por QR, desbloqueando recién en ese momento los archivos originales en alta resolución.
- Académica y Tecnológica: Aplica una arquitectura cliente-servidor escalable y control de versiones distribuido, sentando las bases de un producto de software real.

3. Objetivo general y objetivos específicos

3.1 Objetivo General

Desarrollar una aplicación web para la gestión integral de servicios fotográficos (VisualSync) que permita exhibir el portafolio profesional, administrar reservas de paquetes, validar pagos mediante comprobantes QR y realizar la entrega segura de galerías digitales en alta resolución.

3.1.2 Objetivos Específicos

- Diseñar e implementar un catálogo público interactivo para la exhibición del portafolio fotográfico y paquetes de servicios.
- Estructurar un módulo de reservas que permita a los clientes solicitar fechas de sesión y adjuntar comprobantes de pago mediante código QR.
- Construir un panel de administración para la gestión de reservas, verificación de pagos y carga de fotografías por evento.
- Implementar un mecanismo de control de acceso que habilite la descarga de imágenes en alta resolución exclusivamente tras la confirmación administrativa del pago.
- Configurar la arquitectura base del servidor backend y su versionamiento continuo en GitHub.

4. Usuarios y actores

Tabla 1: Usuarios y Actores

Actor	**Rol en el Sistema**	**Responsabilidades y Acciones Principales**
Visitante (Público)	Usuario no autenticado	Explorar el portafolio público, visualizar detalles/precios de los paquetes fotográficos y registrarse en la plataforma.

Cliente	Usuario autenticado	Solicitar reservas de paquetes seleccionando fecha/hora, subir comprobantes de pago QR, visualizar su galería privada de fotos y descargar los archivos en alta resolución (una vez validado su pago).
Administrador (Fotógrafo)	Gestor del sistema	Administrar el catálogo de paquetes y portafolio, aprobar o rechazar reservas, validar comprobantes QR recibidos, crear galerías privadas por cliente y subir las fotografías procesadas.

5. requerimientos funcionales y no funcionales

5.1 requerimientos Funcionales (RF):

- RF-01: El sistema deberá facilitar al Administrador (fotógrafo) la posibilidad de administrar el catálogo de paquetes de servicios, incluyendo la creación, modificación de precios, descripciones y estados.
- RF-02: El sistema deberá ofrecer a los Clientes la opción de hacer una reserva eligiendo un paquete, así como la fecha y hora del evento.
- RF-03: El sistema debe contar con una interfaz que permita al Cliente cargar una imagen de su comprobante de pago QR o transferencia.
- RF-04: El sistema debe autorizar al Administrador a modificar el estado de un pago (Pendiente, Aprobado, Rechazado).
- RF-05: El sistema debe impedir que los clientes descarguen fotografías en alta resolución si su estado de pago no es Aprobado.
- RF-06: El sistema deberá permitir al Administrador crear galerías privadas y cargar lotes de fotografías asociados a una reserva y a su cliente.

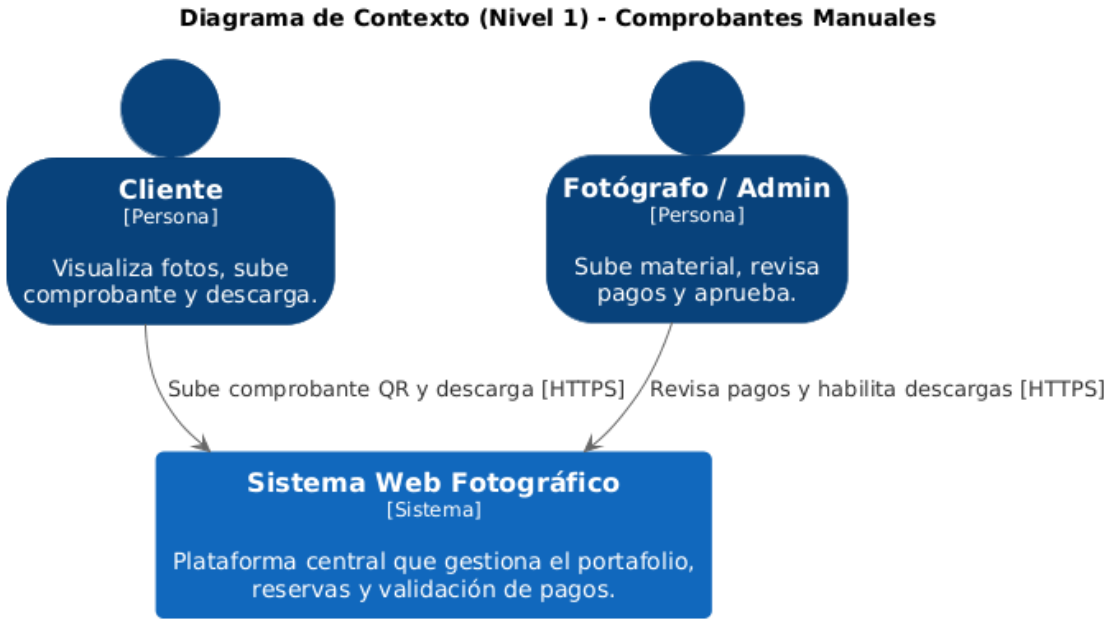
5.1.2 Requerimientos No Funcionales (RNF):

- RNF-01 (Seguridad): Las claves de acceso de los usuarios necesitan ser guardadas de manera encriptada (Hash) en la base de datos. Las áreas administrativas deben exigir autenticación.
- RNF-02 (Rendimiento): La respuesta de la API para búsquedas de texto no debe tardar más de 2 segundos.
- RNF-03 (Usabilidad): La plataforma debe ser adaptable, ajustándose a dispositivos móviles, tabletas y ordenadores de sobremesa.
- RNF-04 (Mantenibilidad): El código tiene que implementar control de versiones usando Git y debe estar hospedado en GitHub, separando la lógica del frontend y del backend.

6. Modelado C4

Nivel 1: Diagrama de Contexto

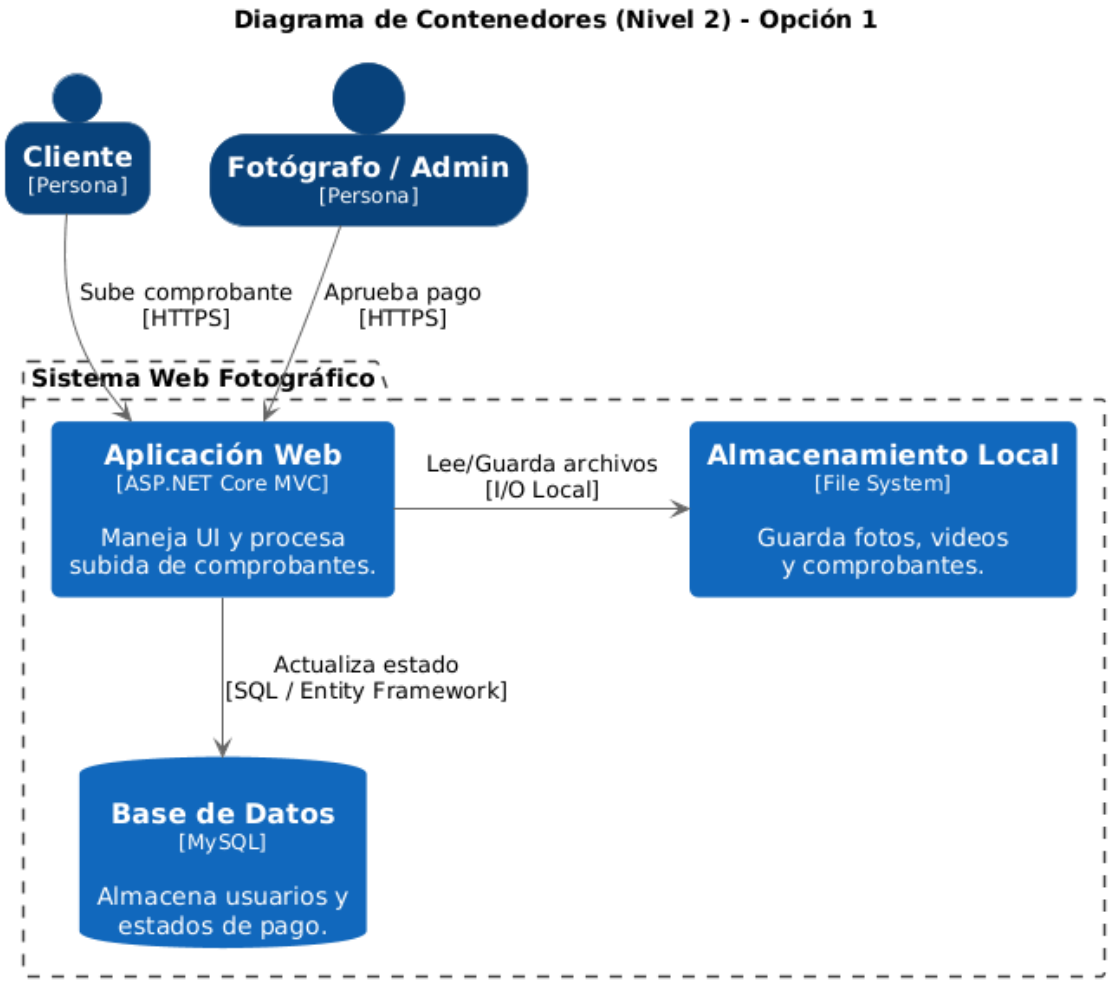
Ilustración 1: Nivel 1: Diagrama de Contexto



Este nivel presenta las interacciones principales desde una perspectiva general. El Cliente se conecta con VisualSync para ver sus fotografías, subir su comprobante de pago por QR y descargar su material, mientras que el Fotógrafo/Administrador sube las fotografías, revisa los comprobantes y habilita las descargas

Nivel 2: Diagrama de Contenedores

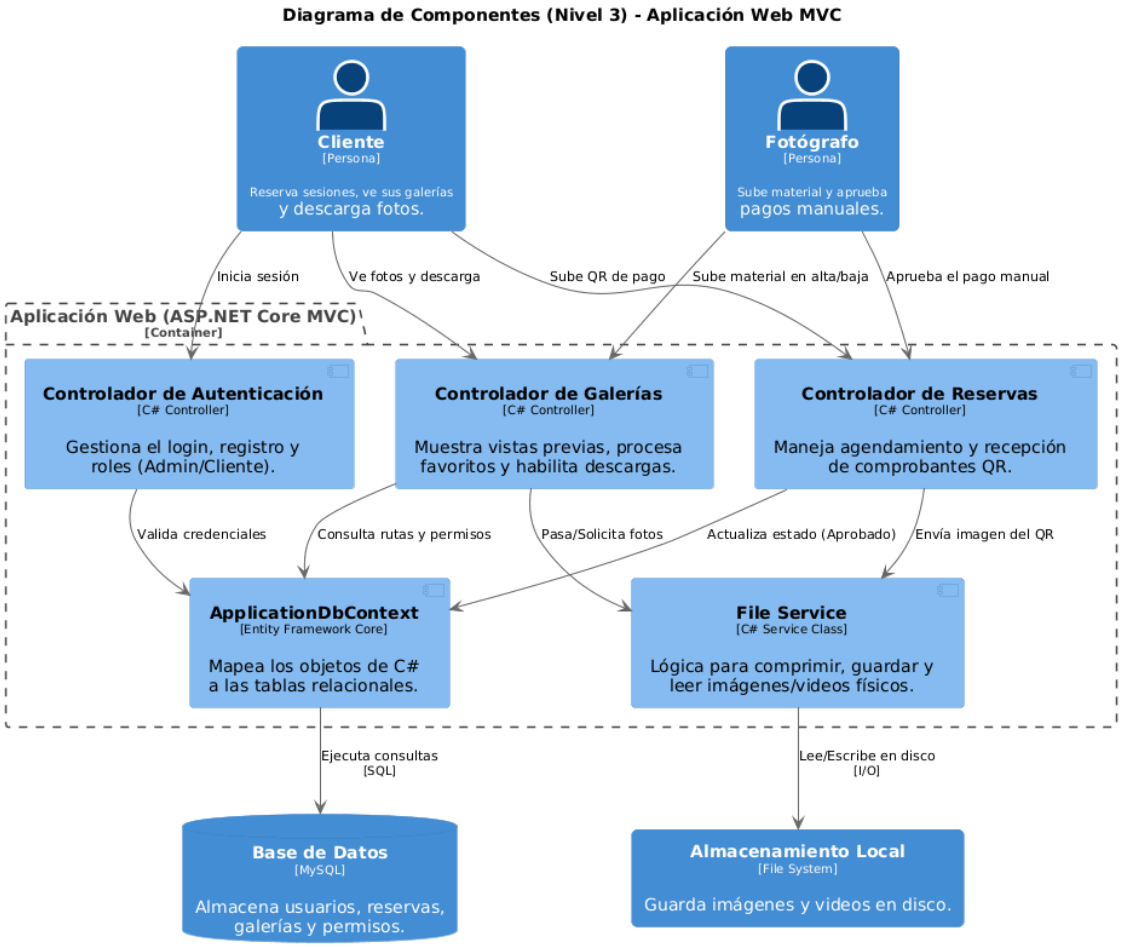
Ilustración 2: Nivel 2: Diagrama de Contenedores



Este nivel describe los contenedores que componen el sistema. La Aplicación Web (ASP.NET Core MVC) gestiona la interfaz y la subida de comprobantes, se comunica con una Base de Datos (MySQL) para registrar y actualizar los estados de pago, y con un Almacenamiento Local (File System) donde se guardan físicamente las fotografías y los comprobantes.

Nivel 3: Diagrama de Componentes

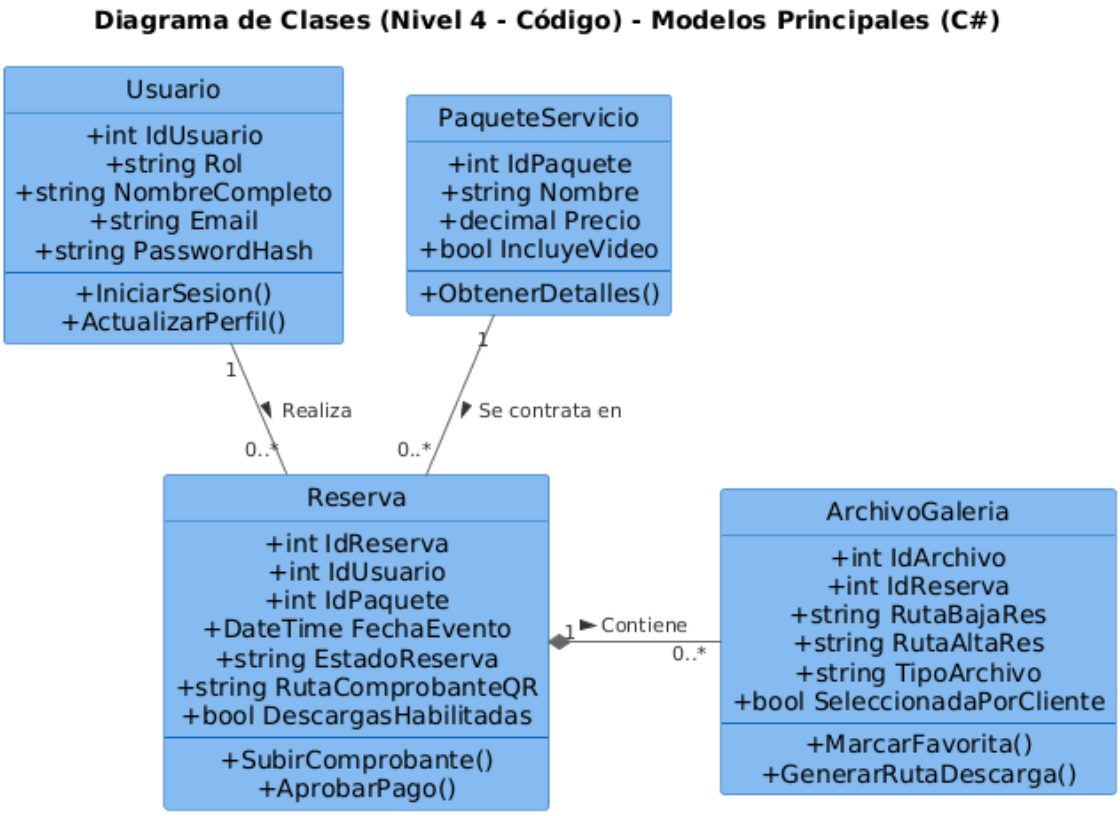
Ilustración 3: Nivel 3: Diagrama de Componentes



Este nivel explora el interior del paquete de la Aplicación Web, separándolo en controladores dedicados: el Controlador de Inicio de Sesión (administra accesos y roles), el Controlador de Imágenes (presenta vistas y permite descargas) y el Controlador de Citas (se encarga de la programación y el manejo de códigos QR).

Nivel 4: Diagrama de Código (Clases)

Ilustración 4: Nivel 4: Diagrama de Código (Clases)



Este nivel detalla los componentes internos de la Aplicación Web. Está organizada en controladores con responsabilidades separadas: el Controlador de Autenticación (gestiona el inicio de sesión, el registro y los roles), el Controlador de Galerías (muestra las vistas previas y controla las descargas) y el Controlador de Reservas (gestiona el agendamiento y la recepción de comprobantes QR). Estos se apoyan en el servicio de archivos (File Service) y en el contexto de Entity Framework Core (ApplicationDbContext) para acceder al almacenamiento y a la base de datos.

7.Historias de Usuario

Para el desarrollo de VisualSync se definen los requerimientos de interacción mediante Historias de Usuario, lo que permite centrar la funcionalidad en el valor que se entrega a cada actor (Cliente y Administrador/Fotógrafo).

7.1 Estados del sistema

Tabla 2: Estados del Sistema

Campo	**Valores**
EstadoReserva	Solicitada, Confirmada, Finalizada, Cancelada
EstadoPago	Sin comprobante, Pendiente de validación, Aprobado, Rechazado

7.1.2 Módulo 0: Acceso

Tabla 3: Registro e Inicio de Sesion

ID	**Título**	**Descripción (Como...Quiero… Para…)**	**Criterios de aceptación**	**Prioridad**
HU-07	Registro e inicio de sesión	Como cliente, quiero registrarme e iniciar sesión, para reservar y acceder a mis galerías de forma segura.	1. El formulario de registro pide nombre completo, correo y contraseña. 2. El correo no puede estar repetido. 3. La contraseña se almacena cifrada (hash). 4. El sistema distingue los roles Cliente y Administrador/Fotógrafo, y cada uno solo ve sus opciones.	Alta

7.1.3 Módulo 1: Catálogo y Reservas

Tabla 4: Catálogo y Reservas

ID	Título	Descripción (Como… Quiero… Para…)	Criterios de aceptación	Prioridad
HU-01	Exploración de paquetes	Como cliente potencial, quiero ver los paquetes fotográficos disponibles con sus precios y detalles, para elegir el servicio que mejor se adapte a mi evento.	1. El sistema muestra una lista de paquetes activos. 2. Cada paquete detalla precio, duración, cantidad de fotos, si incluye video y descripción. 3. Cada paquete tiene un botón “Reservar”.	Alta
HU-02	Solicitud de reserva	Como cliente con sesión iniciada, quiero seleccionar una fecha en el calendario y registrar los datos de mi evento, para agendar mi sesión fotográfica.	1. El formulario pide fecha del evento, lugar y paquete. 2. El sistema valida que la fecha no esté ocupada por una reserva confirmada. 3. Al enviar, la reserva queda en estado “Solicitada”. 4. La fecha se valida de nuevo cuando el administrador aprueba el pago.	Alta

7.1.4 Módulo 2: Pagos y Validación QR

Tabla 5: Pagos y Validacion QR

ID	Título	Descripción (Como… Quiero… Para…)	Criterios de aceptación	Prioridad
HU-03	Carga de comprobante QR	Como cliente, quiero adjuntar una imagen de mi comprobante de transferencia bancaria o pago QR, para confirmar mi reserva.	1. El sistema permite subir archivos .jpg, .jpeg o .png. 2. El tamaño máximo es de 5 MB. 3. El sistema valida el tipo real del archivo, no solo la extensión. 4. Tras la subida, el estado de pago cambia a “Pendiente de validación”. 5. Si el pago es rechazado, el cliente puede subir un nuevo comprobante.	Alta
HU-04	Validación de pagos	Como administrador, quiero visualizar los comprobantes subidos por los clientes, para aprobar o rechazar el pago manualmente.	1. El panel muestra la lista de reservas con pago pendiente de validación. 2. Al hacer clic en el comprobante, se abre en tamaño legible. 3. El administrador tiene botones para	Alta

			cambiar el estado de pago a “Aprobado” o “Rechazado”. 4. Al rechazar, se registra un motivo que el cliente puede ver. 5. Al aprobar, la reserva pasa a “Confirmada” y se habilitan las descargas.	

7.1.5 Módulo 3: Entrega de Material (Galerías)

Tabla 6: Entrega del Material

ID	**Título**	**Descripción (Como... Quiero... Para...)**	**Criterios de aceptación**	**Prioridad**
HU-05	Carga de fotografías	Como administrador, quiero subir lotes de fotografías a la galería privada de una reserva específica, para entregar el trabajo final al cliente.	1. El sistema permite subir múltiples imágenes (.jpg, .png) al mismo tiempo. 2. Las imágenes se asocian automáticamente al IdReserva correspondiente. 3. Por cada foto, el sistema genera una vista previa ligera de baja resolución con marca de agua. 4. La versión original se	Alta

			conserva como alta resolución.	
HU-06	Descarga condicionada	Como cliente, quiero acceder a mi galería y descargar mis fotos en alta resolución, para tener el recuerdo digital de mi evento.	1. El cliente ve miniaturas con marca de agua y baja resolución de sus fotos. 2. El botón “Descargar alta resolución” solo está activo si el estado de pago de esa reserva es “Aprobado”. Si es “Pendiente” o “Rechazado”, el botón está bloqueado. 3. El servidor rechaza la descarga si se intenta acceder por URL directa sin pago aprobado. 4. Un cliente solo puede ver las galerías de sus propias reservas.	Alta

7.1.6 Trabajo futuro (fuera del alcance actual)

Tabla 7: trabajo para el futuro

ID	**Título**	**Descripción**	**Prioridad**
HU-08	Selección de favoritas	Como cliente, quiero marcar mis fotos favoritas, para identificar las que más me gustan.	Baja

HU-09	Gestión de paquetes	Como administrador, quiero crear, editar y desactivar paquetes desde el sistema, para mantener el catálogo actualizado sin tocar la base de datos.	Baja
HU-10	Notificaciones	Como cliente, quiero recibir un aviso cuando mi pago sea aprobado o mi galería esté lista, para enterarme sin tener que revisar el sistema.	Baja

8. Arquitectura propuesta

Para VisualSync se propone una arquitectura cliente-servidor en capas, implementando el patrón MVC (Modelo-Vista-Controlador) en el servidor mediante ASP.NET Core. Esta separación de responsabilidades facilita el mantenimiento, la escalabilidad y la seguridad del sistema.

8.1 Capa de presentación (Cliente / Vistas)

- Descripción: interfaz que opera en el navegador del visitante (Usuario o administrador/fotografo), creada a partir de las Vistas del servidor.
- Responsabilidad: registrar las acciones (formularios, reservas, envío de comprobantes QR), exhibir el inventario de paquetes y las galerías, y mostrar las respuestas del servidor.
- Comunicación: solicitudes HTTP (GET, POST) dirigidas a las direcciones del servidor.

8.1.2 Capa de lógica de aplicación (Servidor / Backend)

Desarrollada utilizando ASP. NET Core MVC (C#). Está conformada por:

- Controladores: gestionan las solicitudes y llevan a cabo las normativas del negocio. Ejemplos incluyen: autenticación y asignación de roles, administración de reservas y regulación de descargas, que solo se habilitan cuando el pago se encuentra en estado "Aprobado".
- Modelos: son clases en C# (Usuario, PaqueteServicio, Reserva, ArchivoGaleria) que simbolizan las entidades dentro del sistema.

- Servicio de archivos: almacena y recupera recibos y fotos en el disco, verifica el tipo y tamaño de los archivos, y crea vistas previas ligeras con marca de agua.
- Acceso a datos: Entity Framework Core (ApplicationDbContext) enlaza los modelos con las tablas en la base de datos relacional.

8.1.3 Capa de datos

- Base de datos: MySQL, que retiene de manera continua información sobre usuarios, paquetes, reservas y las trayectorias de los documentos.
- Almacenamiento local (Sistema de Archivos): almacena de manera tangible los recibos y las imágenes en calidad alta y baja.

9. Modelo de datos inicial

El diseño del esquema relacional (MySQL) se lleva a cabo utilizando la metodología Code-First con Entity Framework Core, empleando el conector Pomelo para MySQL. Este modelo incluye cuatro entidades clave que aseguran la correcta gestión de reservas, pagos y la entrega segura de fotografías. Las imágenes no se almacenan directamente en la base de datos, sino que se guardan en el sistema de archivos del servidor; solamente se registran sus rutas en la base de datos.

9.1 Tabla: Usuarios

Almacena las credenciales y datos de los clientes y administradores.

Tabla 8: Usuarios

Campo	**Tipo**	**Restricciones**	**Descripción**
IdUsuario	int	PK, autoincremental	Identificador único
Rol	string	Obligatorio	“Admin” (administrador/fotógrafo) o “Cliente”

NombreCompleto	string	Obligatorio	Nombre y apellidos
Email	string	Obligatorio, único	Correo usado para iniciar sesión
PasswordHash	string	Obligatorio	Contraseña almacenada como hash

9.1.2 Tabla: PaquetesServicio

Catálogo de servicios fotográficos que ofrece el estudio.

Tabla 9: Paquetes Servicios

Campo	**Tipo**	**Restricciones**	**Descripción**
IdPaquete	int	PK, autoincremental	Identificador único
Nombre	string	Obligatorio	Título (ej. “Boda Premium”)
Descripcion	string	Obligatorio	Detalle del servicio
Precio	decimal	Obligatorio, mayor a 0	Costo del servicio
Duracion	int	Obligatorio	Duración de la sesión en horas
CantidadFotos	int	Obligatorio	Fotos incluidas
IncluyeVideo	bool	Obligatorio	Indica si incluye filmación
Activo	bool	Obligatorio, por defecto true	Si se muestra en el catálogo

9.1.3 Tabla: Reservas

Tabla central que vincula al cliente con el servicio solicitado y registra la fecha, el estado y el comprobante de pago.

Tabla 10: Revervas

Campo	**Tipo**	**Restricciones**	**Descripción**
IdReserva	int	PK, autoincremental	Identificador único
IdUsuario	int	FK → Usuarios	Cliente que reserva
IdPaquete	int	FK → PaquetesServicio	Servicio contratado
FechaEvento	DateTime	Obligatorio	Fecha y hora de la sesión
LugarEvento	string	Obligatorio	Ubicación del evento
EstadoReserva	string	Obligatorio	Solicitada, Confirmada, Finalizada, Cancelada
EstadoPago	string	Obligatorio	Sin comprobante, Pendiente de validación, Aprobado, Rechazado
RutaComprobanteQR	string	Opcional	Ruta de la imagen del comprobante

MotivoRechazo	string	Opcional	Motivo si el pago fue rechazado
DescargasHabilitadas	bool	Por defecto false	Se activa automáticamente al aprobar el pago

9.1.4 Tabla: ArchivosGaleria

Registra las fotografías de cada reserva mediante rutas hacia el almacenamiento local.

Tabla 11: Archivos Galeria

Campo	**Tipo**	**Restricciones**	**Descripción**
IdArchivo	int	PK, autoincremental	Identificador único
IdReserva	int	FK → Reservas	Reserva a la que pertenece
RutaBajaRes	string	Obligatorio	Vista previa con marca de agua
RutaAltaRes	string	Obligatorio	Original en máxima calidad, guardado fuera de la carpeta pública y entregado solo si el pago está aprobado
TipoArchivo	string	Obligatorio	Formato (“.jpg”, “.png”)
SeleccionadaPorCliente	bool	Por defecto false	Favorita del cliente (uso previsto en trabajo futuro)

9.1.5 Relaciones (multiplicidad)

- Usuario - Reserva (1: N): un usuario puede tener varias reservas; cada reserva pertenece a un solo usuario.
- Paquete Servicio - Reserva (1: N): un paquete puede contratarse en muchas reservas.
- Reserva - Archivo Galería (1:0...N): una reserva puede tener cientos de archivos, o ninguno hasta que el fotógrafo suba el material. Si se elimina la reserva, se eliminan sus archivos asociados.

10. Prototipo de interfaces

La creación de las interfaces de VisualSync se enfoca en la facilidad de uso y en la presentación visual, utilizando imágenes y un diseño que se ajusta tanto a móviles como a PC. Esto se lleva a cabo con vistas Razor del framework ASP. NET Core y Bootstrap. La secuencia de las pantallas está estructurada en función del rol del usuario, ya sea Cliente o Administrador/Fotógrafo, y consta de seis vistas esenciales.

10.1. Vista pública: Catálogo de servicios (HU-01)

Propósito: exhibir los servicios del fotógrafo y atraer clientes.

Elementos:

- Barra de navegación con “Inicio”, “Servicios” y “Login”. Si hay sesión iniciada, muestra “Mis reservas” y “Cerrar sesión”.
- Paquetes en tarjetas (cards) con nombre, precio, duración, cantidad de fotos, si incluye video y descripción.
- Botón “Reservar ahora” en cada paquete.

Ilustración 5: Vista pública: Catálogo de servicios



10.1.2 Vista pública: Registro e inicio de sesión (HU-07)

Propósito: permitir el acceso seguro al sistema.

Elementos:

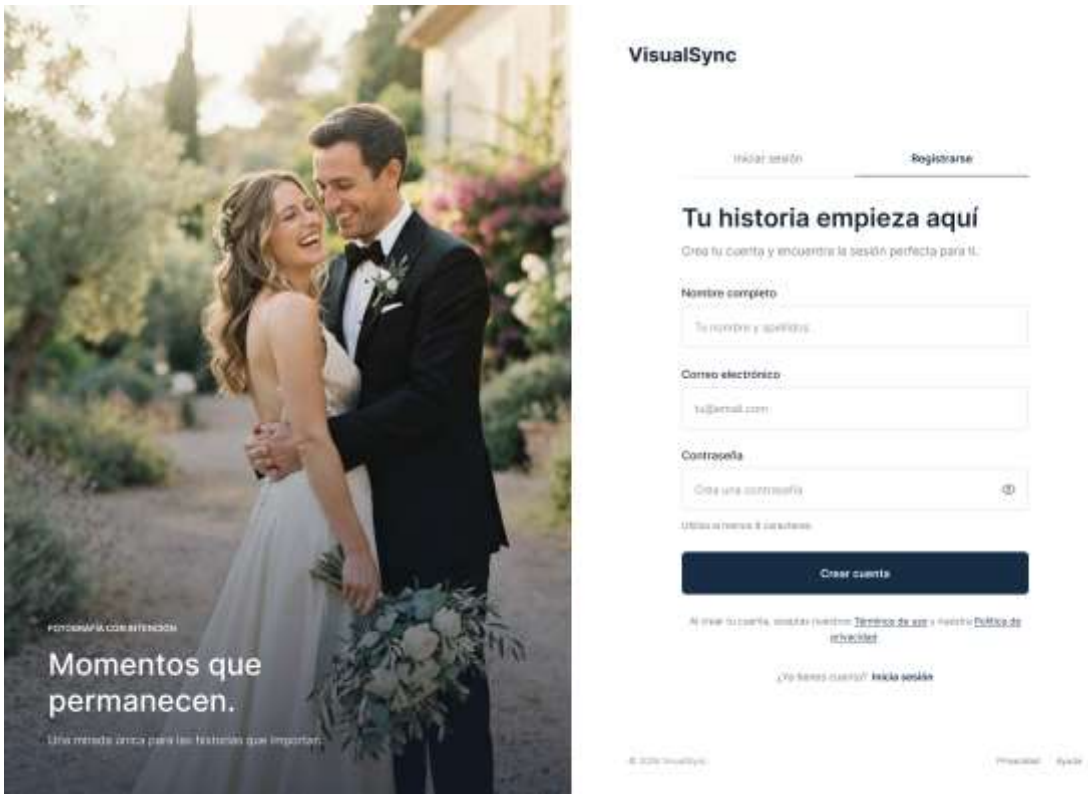
- Formulario de registro: nombre completo, correo y contraseña.
- Formulario de inicio de sesión: correo y contraseña.
- Mensajes de validación, como correo ya registrado o credenciales incorrectas.

Ilustración 6: Vista pública: Inicio de Sesion



Prototipo de interfaz para el inicio de sesión, mostrando la validación de credenciales incorrectas.

Ilustración 7: Vista pública: Registrarse



Prototipo de interfaz para el registro de nuevos usuarios.

10.1.3 Vista de cliente: Solicitud de reserva y pago (HU-02, HU-03)

Propósito: capturar los datos del evento y el comprobante de pago.

Elementos:

- Resumen del paquete seleccionado.
- Calendario interactivo para elegir fecha y hora, con las fechas ocupadas bloqueadas.
- Campo para el lugar del evento.
- Sección de pago con el código QR estático del fotógrafo.
- Zona de carga del comprobante (arrastrar y soltar, o examinar), con formatos .jpg, .jpeg y .png de hasta 5 MB.
- Botón “Enviar solicitud de reserva”.

10.1.4 Vista de cliente: Mis reservas (HU-03)

Propósito: consultar el estado de cada reserva y su pago.

Elementos:

- Lista de reservas con paquete, fecha, estado de reserva y estado de pago.
- Si el pago fue rechazado, muestra el motivo y un botón para subir un nuevo comprobante.
- Acceso a la galería de las reservas que la tengan.

10.1.5 Vista de cliente: Galería privada y descargas (HU-06)

Propósito: entregar el material fotográfico de forma segura.

Elementos:

- Cuadrícula (grid) con las fotos en baja resolución y marca de agua.
- Etiqueta del estado del pago: “Pendiente de validación” (amarillo), “Aprobado” (verde), “Rechazado” (rojo) o “Sin comprobante” (gris).
- Botón “Descargar alta resolución” por foto: bloqueado (gris, con ícono de candado) si el pago no está aprobado, y habilitado si lo está.

10.1.6 Vista de administrador: Panel de gestión (HU-04, HU-05)

Propósito: control operativo por parte del fotógrafo.

Elementos:

- Menú lateral con acceso a “Reservas”.
- Tabla de reservas ordenadas por fecha, con filtro por estado de pago.
- Modal de validación: al hacer clic en el comprobante se abre la imagen ampliada con los botones “Aprobar pago” (confirma la reserva y habilita las descargas) y “Rechazar” (solicita un motivo).
- Dentro del detalle de cada reserva, formulario de carga múltiple de fotografías, que genera automáticamente las vistas previas con marca de agua.

11. Tecnologías seleccionadas y justificación

Para el desarrollo de VisualSync se seleccionó un stack basado en el ecosistema .NET y herramientas de código abierto, con el objetivo de encontrar un balance entre eficiencia, protección y simplicidad en el mantenimiento.

11.1.1 Frontend (capa de presentación)

- HTML5, CSS3 y Bootstrap 5: organizan y dan aspecto a las interfaces. Bootstrap facilita el desarrollo gracias a su sistema de rejilla y sus elementos, asegurando que el diseño se adapte tanto a dispositivos móviles como de escritorio.
- JavaScript: se utiliza en el calendario de selección de fechas, bloqueando aquellas que ya están ocupadas, y en la zona de carga de documentos mediante arrastrar y soltar.
- Razor Views (. cshtml): es un motor de plantillas que forma parte de ASP. NET Core. Permite integrar C# dentro del HTML para crear vistas dinámicas en el servidor, como las listas de paquetes o las galerías.

11.1.2 Backend (lógica de aplicación)

- C# y ASP. NET Core MVC: es el lenguaje y el marco principal. Proporciona un rendimiento excelente y admite de forma nativa el patrón MVC, asegurando un código que es limpio y escalable.
- ASP. NET Core Identity: se encarga de la gestión de registros, inicios de sesión, roles (Admin y Cliente) y el almacenamiento seguro de contraseñas mediante hash.
- Entity Framework Core: un ORM que permite la estrategia Code-First (creación de la base de datos desde clases en C#), simplificando las operaciones CRUD y disminuyendo el riesgo de inyecciones SQL mediante el uso de consultas parametrizadas.
- SixLabors. ImageSharp: biblioteca para el tratamiento de imágenes que crea vistas previas ligeras con marca de agua a partir de imágenes originales.

11.1.3 Base de datos y almacenamiento

- MySQL: un gestor de bases de datos relacional, gratuito, establecido y ampliamente accesible en servicios de alojamiento. Se conecta con EF Core a través del proveedor Pomelo, asegurando la integridad de las relaciones entre usuarios, reservas y pagos.
- Sistema de archivos del servidor: guarda fotografías y recibos en el disco en vez de en la base de datos (BLOB). Esto mantiene la base de datos liviana y facilita las copias de seguridad. Las imágenes originales de alta resolución se almacenan fuera de la carpeta pública y se entregan solo a través de un controlador que valida el pago aprobado.

11.1.4 Control de versiones y herramientas

- Git y GitHub: permiten el control de versiones, la copia de seguridad del código y el seguimiento del progreso mediante commits.
- Visual Studio Code y SDK de. NET : plataforma de desarrollo para compilar, ejecutar y depurar el proyecto.
- PlantUML y Figma: utilizados para crear los diagramas de la arquitectura y los prototipos de la interfaz.

12. Estructura inicial de la API

Dado que VisualSync utiliza ASP. NET Core MVC, sus puntos finales pueden devolver vistas HTML (Razor), documentos (comprobantes y fotos) o información en formato JSON (como las fechas reservadas en el calendario) según sea necesario. Las rutas especifican la forma en que el navegador interactúa con el servidor.

Tabla 12: Estructura de carpetas

VisualSync/
Controllers/
Models/
ViewModels/
Data/
Services/
Views/
Migrations/
wwwroot/
Storage/ (fuera de wwwroot, acceso solo vía controlador)
appsettings.json
Program.cs

- Controllers/: controladores que gestionan las peticiones HTTP: HomeController, AuthController, ReservasController, GaleriasController y AdminController.
- Models/: clases C# que representan las entidades del sistema: Usuario.cs, PaqueteServicio.cs, Reserva.cs y ArchivoGaleria.cs.
- ViewModels/: clases que agrupan los datos que cada vista muestra o recibe.
- Data/: contiene ApplicationDbContext.cs, el contexto de Entity Framework Core.

- Services/: lógica reutilizable, como FileService.cs, que valida, guarda y lee archivos del disco y genera las vistas previas con marca de agua.
- Views/: interfaces Razor (.cshtml), organizadas en subcarpetas según el controlador.
- Migrations/: historial de cambios del esquema generado por Code-First.
- wwwroot/: directorio público con archivos estáticos (CSS de Bootstrap, JavaScript, logotipos).
- Storage/: directorio privado, fuera de wwwroot, donde se guardan comprobantes y fotografías. Sus archivos solo se entregan a través de controladores que verifican la sesión y el estado del pago.
- appsettings.json: configuración de la aplicación, incluida la cadena de conexión a MySQL. Las credenciales sensibles se guardan con User Secrets y no en el repositorio.
- Program.cs: punto de entrada donde se configuran los servicios, Entity Framework Core, la autenticación, la autorización por roles y el enrutamiento.

Tabla 13:de rutas iniciales

Método	Ruta	Controlador y acción	Acceso	Propósito
GET	/	HomeController.Index()	Público	Landing page y catálogo de paquetes (HU-01)
GET / POST	/Auth/Login	AuthController.Login()	Público	Formulario e inicio de sesión (HU-07)
GET / POST	/Auth/Registro	AuthController.Registro()	Público	Creación de cuenta de cliente (HU-07)

POST	/Auth/Logout	AuthController.Logout()	Autenticado	Cierra la sesión
GET	/Reservas/Solicitar/{id}	ReservasController.Solicitar()	Cliente	Formulario de reserva y pago QR para el paquete {id} (HU-02)
GET	/Reservas/FechasOcupadas	ReservasController.FechasOcupadas()	Cliente	Devuelve las fechas no disponibles para el calendario
POST	/Reservas/Crear	ReservasController.Crear()	Cliente	Registra la reserva y recibe el comprobante (HU-02, HU-03)
POST	/Reservas/SubirComprobante/{id}	ReservasController.SubirComprobante()	Cliente	Reemplaza el comprobante de una reserva con pago rechazado (HU-03)

GET	/Reservas/MisReservas	ReservasController.MisReservas()	Cliente	Lista las reservas del cliente con su estado y pago
GET	/Galerias/Ver/{id}	GaleriasController.Ver()	Cliente (dueño)	Cuadrícula de fotos en baja resolución de la reserva {id} (HU-06)
GET	/Galerias/Miniatura/{id}	GaleriasController.Miniatura()	Cliente (dueño)	Entrega la vista previa con marca de agua del archivo {id}
GET	/Galerias/Descargar/{id}	GaleriasController.Descargar()	Cliente (dueño)	Entrega la foto original {id} solo si el pago está “Aprobado” (HU-06)
GET	/Admin/Dashboard	AdminController.Dashboard()	Admin	Panel con reservas y pagos pendientes (HU-04)

GET	/Admin/Comprobante/{id}	AdminController.Comprobante()	Admin	Muestra el comprobante de la reserva {id} en tamaño legible
POST	/Admin/AprobarPago/{id}	AdminController.AprobarPago()	Admin	Aprueba el pago, confirma la reserva y habilita las descargas (HU-04)
POST	/Admin/RechazarPago/{id}	AdminController.RechazarPago()	Admin	Rechaza el pago y registra el motivo (HU-04)
GET / POST	/Admin/Reserva/{id}	AdminController.Reserva()		

Tabla 14: Detalle de peticiones y respuestas

Endpoint	Recibe	Respuesta exitosa	Errores

POST /Auth/Registro	Formulario: nombreCompleto, email, password	302 redirige a /Auth/Login	400 si faltan datos o el correo ya existe
POST /Auth/Login	Formulario: email, password	302 redirige según el rol (Cliente: /Reservas/MisReservas; Admin: /Admin/Dashboard)	401 si las credenciales son incorrectas
GET /Reservas/FechasOcupadas	Nada (usuario autenticado)	200 con JSON: ["2026-11-15", "2026-11-22"]	401 si no hay sesión
POST /Reservas/Crear	Formulario: idPaquete, fechaEvento, lugarEvento y archivo del comprobante (.jpg, .jpeg o .png, máx. 5 MB)	302 redirige a /Reservas/MisReservas; la reserva queda “Solicitada” y el pago “Pendiente de validación”	400 si falta un dato o el archivo no es válido; 409 si la fecha ya está ocupada; 401 sin sesión
POST /Reservas/SubirComprobante/{id}	Archivo del comprobante	302 redirige a /Reservas/MisReservas; el pago vuelve a “Pendiente de validación”	400 si el archivo no es válido; 403 si la reserva no es suya o el

			pago no está rechazado
GET /Galerias/Miniatura/{id}	id del archivo	200 con la imagen en baja resolución y marca de agua	403 si la reserva no es del usuario; 404 si no existe
GET /Galerias/Descargar/{id}	id del archivo	200 con el archivo original	401 sin sesión; 403 si el pago no está aprobado o la reserva no es suya; 404 si no existe
POST /Admin/AprobarPago/{id}	id de la reserva	302 redirige a /Admin/Dashboard; pago “Aprobado”, reserva “Confirmada” y descargas habilitadas	403 si no es Admin; 404 si no existe; 409 si la fecha ya fue confirmada para otra reserva

POST /Admin/RechazarPago/{id}	Formulario: motivo	302 redirige a /Admin/Dashboard; pago “Rechazado”	400 si no hay motivo; 403 si no es Admin
POST /Admin/Reserva/{id}	Varias imágenes (.jpg, .png, máx. 25 MB cada una)	302 redirige al detalle de la reserva; se crean los registros de ArchivoGaleria con sus vistas previas	400 si algún archivo no es válido; 403 si no es Admin

13.1 Repositorio GitHub

Para asegurar la protección del código fuente, el respaldo y la supervisión de los progresos de VisualSync, se emplea un sistema de control de versiones utilizando Git, con almacenamiento en la nube a través de GitHub.

13.1.1 Alojamiento del proyecto

- Plataforma utilizada: GitHub.
- Repositorio: https://github.com/david08234/VisualSyncMVC.git
- Contenido inicial: README. md que incluye la explicación del proyecto, los requisitos necesarios y las instrucciones de ejecución, junto con el archivo. gitignore específico para. NET.

13.1.2 Estrategia de ramas

Se establece un modelo simplificado basado en ramas de funciones:

- main: rama fundamental y estable. Solo incluye código que funciona y ha sido verificado.
- feature/nombre-funcionalidad: ramas temporales generadas desde main para cada tarea individual (por ejemplo, feature/hu-02-reserva). Tras finalizar, se integran en main y se eliminan.

13.1.3 Convención de commits

Los mensajes de los commits se asocian con las Historias de Usuario mediante prefijos específicos. Ejemplos:

- feat(HU-01): añadir visualizaciones del catálogo de paquetes
- fix(HU-03): subsanar la validación del tamaño del recibo QR

13.1.4 Archivos ignorados (. gitignore)

Se aplica la plantilla oficial de. NET para prevenir que se suban archivos binarios o información sensible, que incluye:

- Directorios de compilación y entorno: bin/, obj/ y. vs/.
- Archivos de configuración locales con datos personales: appsettings. Development. json. Las credenciales se administran a través de User Secrets.
- Archivos cargados por los usuarios: la carpeta Storage/, que alberga recibos de pago y fotos de clientes y no debe ser publicada.

14. Conclusiones

El diseño de VisualSync permitió definir, antes de programar, una solución completa para la gestión de reservas, pagos y entrega de fotografías de un estudio fotográfico. Las historias de usuario establecieron los requisitos del flujo completo, y el modelado C4, la arquitectura MVC, el modelo de datos y el mapa de rutas describen un mismo sistema, apoyado en ASP.NET Core MVC, Entity Framework Core y MySQL.

Un punto central del diseño es que la descarga de fotografías en alta resolución solo se habilita con el pago aprobado, y esa condición se verifica en el servidor y no solo en la interfaz. Para que el proyecto sea realizable por una sola persona, se acotó el alcance a las funciones esenciales, y quedan como trabajo futuro las fotos favoritas, la gestión de paquetes, las notificaciones y la entrega de archivos RAW. La siguiente fase consiste en desarrollar el sistema en el repositorio de GitHub por módulos: autenticación, reservas, pagos y galerías.